

An Exploration on The Effects of Parameter Tuning in Ant Colony Optimization in Solving the Travelling Salesman Problem

Krish Sharma
School of Computing
Asia Pacific University
Kuala Lumpur, Malaysia
tp084234@mail.apu.edu.my

Leonardo Fernando
School of Computing
Asia Pacific University
Kuala Lumpur, Malaysia
tp081483@mail.apu.edu.my

Manoj Vishnu
School of Computing
Asia Pacific University
Kuala Lumpur, Malaysia
tp086203@mail.apu.edu.my

Pieter Deli Ho
School of Computing
Asia Pacific University
Kuala Lumpur, Malaysia
tp079048@mail.apu.edu.my

Ruben Vikrando
School of Computing
Asia Pacific University
Kuala Lumpur, Malaysia
tp079847@mail.apu.edu.my

Abstract—*The Traveling Salesman Problem (TSP) is a well-known optimization problem that scales in difficulty as the number of cities grows. To address this problem, metaheuristic algorithms such as Ant Colony Optimization (ACO) are commonly used to produce semi-optimal to optimal solutions within reasonable time. However, the performance of traditional ACO is strongly influenced by parameter sensitivity that are affecting the balance between exploration and exploitation during the search process. This study investigates the impact of different parameter configurations on the behaviour and effectiveness of standard ACO on TSP instances. The approach is done by implementing a basic ACO model and evaluating its performance under different parameter settings using benchmark TSP instances. The findings indicate that parameter selection has a significant impact on solution quality and convergence behaviour. Modifying the parameters values may lead to better or worse solution quality. These demonstrate that performance improvements can be done through manual parameter tuning without any algorithm structural changes or any hybrid approach.*

Keywords— *Ant Colony Optimization, Traveling Salesman Problem, Metaheuristics, Parameter Sensitivity, Exploration, Exploitation*

I. INTRODUCTION

A class of issues known as Combinatorial Optimization Problems (COPs) include selecting the best solution from a finite but incredibly vast collection of potential combinations. Exact solution approaches are unfeasible for big instances of these issues because they are usually characterized by discontinuous solution spaces and quickly growing processing complexity as the problem size increases. The Knapsack Problem, Job Shop Scheduling Problem, Vehicle Routing Problem, and Traveling Salesman Problem are typical instances of COPs (Lawler et al., 1985).

The Traveling Salesman Problem (TSP) is a traditional combinatorial optimization issue where the objective is to find the shortest path that makes exactly one stop in each city before returning to the starting point is the

goal, creating a Hamiltonian circuit. The TSP is categorized as an NP-hard issue despite its straightforward formulation, which means that the computer work needed to find an exact solution grows exponentially as the number of cities increases (Garey & Johnson, 1979). The TSP has been used extensively as a benchmark issue for assessing heuristic and metaheuristic optimization methods because of its difficulty (Applegate et al., 2006).

Ant Colony Optimization (ACO) has garnered a lot of attention among the different metaheuristic methodologies put forth to address the TSP. Based on the foraging habits of actual ants that use pheromone trails to communicate indirectly (Dorigo et al., 1996). Artificial ants use pheromone information and heuristic guidance to design tours probabilistically in the setting of the TSP, which enables high-quality solutions to emerge through collective learning over multiple iterations (Dorigo & Gambardella, 1997).

However, parameter adjustments have a significant impact on how well standard ACO algorithm's function. The balance between exploration and exploitation during the search process is directly impacted by parameters that determine the influence of pheromone trails, heuristic information, pheromone evaporation, and the quantity of ants. Poor solution quality can occur from improper parameter setups that cause early convergence, stagnation, or excessive exploration. Parameter sensitivity is a fundamental weakness of basic ACO techniques, as previous research has shown that fixed parameter values may not always generalize well across different TSP cases (Stützle & Dorigo, 1999).

Given this constraint, improving ACO's practical application to the TSP requires an understanding of how parameter settings affect its behaviour. This work focuses on a conventional ACO framework, investigating the outcome of manual parameter configurations over several experimental runs, as opposed to suggesting sophisticated algorithmic variations or adaptive mechanisms. This paper attempts to reveal how parameter choices impact solution quality and convergence behaviour when solving the Traveling Salesman Problem by examining exploration-

heavy, exploitation-heavy, and balanced parameter configurations.

II. RELATED WORKS

This section talks about the Ant Colony Optimization (ACO) algorithm and the Traveling Salesman Problem (TSP) on subjects like objective, idea, procedures, results, limitations and implications.

TSP's main objective is to determine the shorter possible route that visits every city exactly once and must return to the original starting place. TSP has a great computational complexity, algorithms become more impractical as problem sizes grew. Resulting in search for algorithms that can satisfy or perform better in this problem, this can be done with the use of heuristic and metaheuristic approaches (Dorigo et al., 1996; Dorigo & Gambardella, 1997).

The main use of ACO for TSP is to obtain high-quality approximate solutions within reasonable computational time while maintaining robustness across different TSP instances. Traditional ACO can achieve this by simulating the foraging behaviour of artificial ants that mimic real ants, the artificial ants construct tours based on pheromone trails and heuristic information, typically in the form of distance between cities (Dorigo & Blum, 2005). This baseline idea allows the algorithm to reinforce better and shorter tours as iterations adds up.

In the context of ACO, standard ACO follows a relatively simple sequence of steps. Pheromone values are initially assigned to all edges in the graph. During each iteration, a colony of ants constructs complete tours by probabilistically selecting the next city based on pheromone intensity and heuristic desirability. After completion of the iteration, pheromone updates are applied by reinforcing edges that belong to shorter tours and pheromone evaporation reduces the influence of older less desired solutions. This process is repeated until a termination condition is met, usually by setting a number of maximum iterations that can be done (Dorigo et al., 1996; Dorigo & Gambardella, 1997). Thus, this procedure allows for rich search behaviour driven by parameter settings and iteration dynamics.

Numerous studies report that the results achieved by traditional ACO on TSP are strongly influence by global control parameters rather than the algorithm's structure itself. These parameters such as pheromone importance (α), heuristic importance (β) and pheromone evaporation rate (ρ) directly tip the balance between exploration and exploitation. Empirical (based on observation rather than theory) investigations demonstrate that high pheromone influence acceleration in ants' convergence but also increases the risk of premature stagnation, while higher heuristic value promotes exploration but may result in worse stability (Abid et al., 2023; Zarrinpanjeh et al., 2020). Moderate balance on evaporation rates is often better on preserving solution diversity while still allowing high-quality paths to be reinforced over iterations (Ullrich et al., 2024).

Iteration behaviour has also been seen to play a big role in ACO overall performance. Increasing the number of iterations generally improves solution quality, especially when exploration is preserved during early stages of the search (Zarrinpanjeh et al., 2020). On the other hand, too many iterations count significantly increase the computational cost needed to keep the algorithm going. To further add, poorly tuned parameters may still lead to stagnation despite the longer runs. Large-scale TSP studies indicate that careful coordination between iteration limits and parameter settings is essential for maintaining convergence efficiency (Skinderowicz, 2022; de Oliveria et al., 2021).

Parameter sensitivity is also one of factors that can easily affect the solution quality of the problem. Many studies consistently show that small variations in the parameters can produce large difference in performance outcomes (Nallaperuma et al., 2015; Abid et al., 2023). These findings proves that there are no universal parameter settings that will work for all TSP instances. Each instance will need its own specific parameter settings as graph size and city distributions vary. While some recent works propose adaptive or dynamic parameter strategies, they also confirm that the fundamental behaviour of ACO remains heavily affected by its core parameters (Wang & Han, 2021; Shi et al., 2022).

Despite all the benefits of traditional ACO, it also exhibits several limitations. The algorithm is highly sensitive to parameter selection and often requires extensive trial-and-error rather than theory to achieve semi-optimal or optimal results. For very large TSP instances, computational cost increases rapidly with colony size and iteration count. To further add, although local refinement techniques such as 2-Opt or 3-Opt can improve solution quality, they add computational overhead and must be properly balanced (Stützle & Hoos, 2000; Hao et al., 2023). These limitations indicate that performance improvements must be carefully balanced while maintaining algorithmic simplicity and computational efficiency.

Nevertheless, traditional ACO has been proven to work very well with most TSP instances. Rather than replacing the algorithm with complex hybrids implementations, many studies demonstrate that meaningful performance gains can be achieved by analysing and tuning parameter settings, iteration scheduling and basic local refinement strategies (Dorigo & Blum, 2005; Skinderowicz, 2022). This approach preserves the interpretability and simplicity of standard ACO while enhancing its practical effectiveness.

In conclusion, prior studies shows that ACO remains a solid and flexible algorithm for solving TSP, with its performance largely influenced by control parameters and iteration behaviour. Previous studies have also demonstrated that careful parameter tuning improves convergence speed, solution quality and stability. On the other hand, improper configurations will lead to stagnation

or inefficiency. These findings justify the focus of the present research on systematically studying ACO parameters rather than modifying the algorithm's structure.

III. MATERIALS AND METHODS

The ACO algorithm used in the research was obtained from a GitHub repository authored by Marín (2020). The algorithm is then run using the PyCharm IDE from JetBrains and tested using berlin52, ch130 and kroA100 instances from the TSPLIB standard library.

Marín (2020) outlines the program architecture of the Ant Colony Optimization (ACO) algorithm, which starts off by parameter initialization and random placement of ants into the problem space. Then, each ant builds a solution in a probabilistic, node-per-node manner by utilizing pheromone trails and heuristic information as guides until a complete solution is found. The following formula is used:

$$P_{ij}(t) = \frac{\tau_{ij}^\alpha \eta_{ij}^\beta}{\sum (\tau_{ij}^\alpha \eta_{ij}^\beta)} \quad (1)$$

Where τ (tau) is the amount of pheromones and η (eta) is the inverse of the distance ($1/d$). α (alpha) and β (beta) are the algorithm's parameters. They are used to give more or less weight to the distance or pheromones while selecting the next node.

Pheromone trails are then updated to reinforce the paths of good solutions, where the trail gets further reinforced if more ants follow the exact path. The trails are incremented using this formula:

$$\tau^{(i+1)} = \tau^i + \Delta\tau \quad (2)$$

Where $\Delta\tau$ (delta_tau) is a parameter and i is the iteration number.

Suboptimal path influences are reduced after each iteration in a process called pheromone evaporation. The evaporation formula is:

$$\tau^{(i+1)} = (1 - \rho)\tau^i \quad (3)$$

Where ρ (rho) is the evaporation rate.

Finally, the algorithm stops once the termination condition, which in this case is the maximum number of iterations, have been reached. The basic flow of the program is outlined in a flowchart, shown in Fig. 1.

Parameters for the algorithm are located inside the `aco tsp.py` script, and a default list of values were set as listed in Table I. The parameters tuning process will be conducted manually, in which values of the respective parameters are modified: following the state transition rule for balanced and exploring high exploration and high exploitation. The results of each test are then compared against the default control to evaluate the effects of parameter tuning on the overall performance of the algorithm.

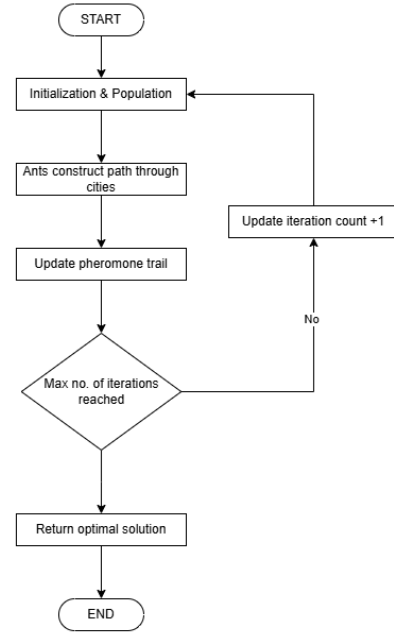


Fig. 1. The flowchart of basic ACO algorithm.

A. Purpose

To observe different behaviours based on parameter changes, as an experimental analysis. To improve the quality of the solution, finding tour lengths that could be close to the known best for each dataset, and adjust it to control the balance between exploration and exploitation, which directly affects the convergence speed, computational cost, and solution quality. (Miranda, 2017).

B. Parameters and Configurations

The code provides 6 parameters that can be modified. Each configuration for certain parameters will produce different outcomes.

Iteration defines how many times the ACO will construct solutions and updates pheromones. This condition is considered the termination condition of the algorithm. It will affect the runtime of the code and the stability of the pheromone trails. This parameter is a simple control over the search depth, where higher iterations generally improve the solution quality. However, having higher iterations can lead to an increase in the runtime linearly, and may cause stagnation if diversity is lost. (Wu et al., 2025).

Colony size specifies the amount of ant that will be used for each iteration. This parameter will affect search diversity and the computational cost. It helps to improve the robustness against local optima, and the probability of finding good regions of the search is greatly increased when more ants are added. However, high colony size is not always good, as it can increase the runtime significantly, and too many ants can lead to noise in pheromone updates. (Wu et al., 2025).

Alpha (α) controls the importance of pheromone trails in the probability of choosing the next city. This parameter will affect the degree of collective learning and the convergence speed. It is a useful parameter to configure, as it encourages reusing good solutions, and accelerates convergence when good paths are found. However, having high Alpha can lead to premature convergence, while lower Alpha can slow the learning and increases the randomness. (Hussein, 2020).

Beta (β) weigh the heuristic information. In TSP, it will be considered as the inverse of the distance. The parameter will affect the greediness of path election and produce early-stage convergence. It helps the ants to build reasonable tours early and reduces the random wandering of the ants. However, having too high of Beta will make the algorithm behave like greedy nearest-neighbour, and will ignore pheromone learning if it is overly dominant. (Hussein, 2020).

Delta Tau ($\Delta\tau$) controls how much pheromone will be deposited after ants has completed a tour. This parameter will affect the speed of reinforcements and strengthens the learning path. It is useful for faster learning of high-quality solutions and will enhance collective memory. However, having larger amount of Delta tau may overpower the evaporation, increasing the risk of stagnation. (Dorigo, M. 2007).

Rho (ρ) determines the evaporation rate of pheromones after each iteration. The parameter will affect the memory retention and the adaptability of the algorithm. It is useful to prevent unlimited pheromone accumulation and will help to escape the local optima. However, having too high of Rho can prevent ants from learning, while having too low Rho can cause stagnation. (Hussein, 2020).

The tested parameter configurations are as follows:

Additional iteration, iteration was changed to see if more feedback cycle will create better results. Iteration will be doubled from the default iteration. Too high of an iteration will produce a longer runtime and might result in premature convergence. (Shen et al., 2025).

Exploration-heavy, ants will rely more on distances and less on pheromone trails. The high evaporation rate makes pheromone information fade quickly. The main parameters, to be changed, will be the alpha, beta, and the evaporation rate. Alpha will be kept low, but not too low as pheromone influence will be too weak which can make reinforcement to be too slow and will require more iteration, often leading to failures in converging within reasonable runtime. Beta will be increased, to encourage distance-based exploration, but will not be too high as to prevent greedy distance bias. Evaporation rate will be kept high in the range of 0.5 - 1.0 to avoid strong reinforcements. (Wang & Han, 2021).

Exploitation-heavy, ants will give much higher weight to pheromone trails, meaning they strongly follow

successful routes. Low evaporation keeps pheromone trails longer. The parameters change will be the opposite of exploration, where alpha, beta, and the evaporation rate will be the only ones changed. Alpha will be higher, but not too high to avoid premature convergence. Beta will be lower, but not too low as distance will have little to no influence, having early random pheromone bias. Evaporation rate will be kept very low, but never zero, so early mistakes will be corrected. And will have a mechanism to avoid local optima. (Wang, Zariphopoulou, & Zhou, 2018).

Balanced exploration and exploitation, this method will achieve a middle ground where ants will explore widely early but also begin exploiting promising paths in later iterations. The parameters here will be kept at a balance value, not too high or too low based on the parameter values that were used in the exploration and exploitation method. This is to see whether the having a balanced of both methods is more reliable compared to just having more bias in each one. (Wang, Zariphopoulou, & Zhou, 2018).

IV. RESULT

The test will be done on popular instances of TSPs: ch130, kroA100, and Berlin52. The parameters and number of iterations used for each type are given in Table I. The simulation results summary of running each case 3 times is listed in Table II.

The results show that exploitation-heavy settings produced lower average tour lengths across all tested TSP instances.

Meanwhile exploration-heavy settings produced a higher variance of solutions, balanced settings providing a stable but non-competitive solution, and number of iterations does not directly correlate to better solution results, while computational time difference apart from increased iterations are negligible

V. DISCUSSIONS

Performance and analysis of each type of parameter configuration will be primarily measured through deviation of improvement or deterioration of distance value from the default tested parameters.

TABLE I PARAMETER VALUES

Oriented Parameter Configuration	Parameters					
	N_{iter}	m	α	β	$\Delta\tau$	ρ
Default	100	25	1.2	1.5	2	0.2
+100 Iterations	200	25	1.2	1.5	2	0.2
Exploration-heavy	100	25	0.8	2.5	2	0.6
Exploitation-heavy	100	25	2.0	1.0	2	0.1
Balanced	100	25	1.0	2.0	2	0.3

TABLE II TEST RESULTS

TSP	Oriented Parameter Configuration	Solution			
		Best	Worst	Mean	Average time (s)
ch130	Default	26605.89	27933.65	27458.56	4.31
	+ 100 Iterations	26581.69	27585.58	27157.48	4.27
	Exploration-heavy	26164.56	27200.87	26756.41	3.77
	Exploitation-heavy	25889.33	27046.57	26354.63	3.87
	Balanced	26978.14	27950.43	27344.51	3.84
kroA100	Default	62188.55	62738.98	62481.41	2.29
	+ 100 Iterations	66131.98	68139.34	67266.49	2.44
	Exploration-heavy	60314.21	68253.69	64697.62	2.15
	Exploitation-heavy	61010.20	64216.42	62933.66	2.19
	Balanced	61615.91	72397.05	66986.85	2.28
berlin52	Default	13929.69	15993.50	14921.50	0.80
	+ 100 Iterations	13929.69	15438.10	14432.49	0.72
	Exploration-heavy	13929.69	15798.01	14975.19	0.64
	Exploitation-heavy	13929.69	14878.47	14245.95	0.72
	Balanced	13929.69	15327.86	14395.74	0.63

A. Interpretation

The results show that ACO performance is highly dependent on its parameters, particularly in the trade-offs between exploration and exploitation. Across all tested TSP instances, most tested parameter orientations provided a more optimal solution compared to the default parameter settings, with only a few showing worse results such as *Balanced* in ch130 and *+100 Iterations* in kroA100 shown in Table 2. The average computational time of each configuration run is shown to have minor differences from the tested instances, meaning that the affected parameters play a limited role in efficiency for small to medium instances, referring to ch130 and kroA100.

B. Exploration vs Exploitation Behaviour

This subsection focuses on the two main configurations of parameters that are exploitation and exploration based. These results showed that exploitation dominates in most areas compared to exploration, having the best solution, lowest worst-case distance, and lowest average distance within the ch130 instance. Meanwhile exploration distinctively provides the best solution compared to other configurations in kroA100 but loses on other factors due to its high variability across runs. This connects to our initial claim that exploitation-heavy parameters influence ants to converge faster to the first found solution, creating a lower average distance. This behaviour is believed to be advantageous for smaller scaled instances such as ch130 but struggle on bigger scaled instances like kroA100 shown by the results.

C. Stagnation in Basic ACO

An observation can be found for the berlin52 instance results, every parameter configuration strategy leads to a fixed distance of 13929.69. This is caused by early stagnation, ants converging to the same local optimum which is common within very small instances. It is a limitation of the basic ACO model itself instead of the parameter configurations, explaining the identical results. This tells us that parameter variation alone is insufficient to overcome a main weakness of ACO. For this reason, more advanced models such as Hybrid-ACO models and local-optimisation improvement models (e.g. 2-Opt, 3-Opt) are adopted in newer studies.

D. Iteration Against Performance

Strategy of increased iterations was shown to be ineffective within the test results, resulting in a worse solution compared to default parameters. With this finding, the author suggests that higher number of iterations does not directly tie to better solutions, instead reinforcing suboptimal paths.

VI. CONCLUSION

This paper explores the effects of parameter tuning in an Ant Colony Optimization algorithm for solving the Traveling Salesman Problem using TSPLIB instances ch130, kroA100 and berlin52.

Across all tested instances, parameter settings that are exploitation-heavy consistently produced a better average

solution quality compared to the default, balanced, and exploration-heavy parameter configurations.

Exploration-heavy parameter configurations achieved the best individual solution in TSP instance kroA100. However, excessive exploration reduces solution stability, showing that exploration is not efficient without the addition of proper exploitation pressure.

Parameter tuning, if done alone, does not have the ability to overcome early stagnation problems, shown in the berlin52 test instance, where all parameter configurations resulted in identical best solutions.

Increasing the number of iterations does not guarantee the improvement of solution quality, and in some cases resulted in an inferior solution quality compared to the default parameter configuration.

The results of this experimental study have demonstrated that the performance of an ACO algorithm is strongly influenced by parameter configuration, in particular the balance between exploration and exploitation.

This research has known limitations. The algorithm used implements the most basic form of ACO, without any implementations of hybrid strategies. The evaluation was only performed on 3 benchmark TSP instances, and each parameter configuration was executed only three times per TSP instance. While the drawn conclusions are valid for the test setup, further experimentation with a more diverse set of parameter configurations and benchmark instances are needed to further validate the observed results.

REFERENCES

- Abid, A., Khan, M. A., & Shah, S. I. (2023). Sensitivity analysis of ant colony optimization parameters. *IEEE Access*, 11, 1–15. <https://doi.org/10.1109/ACCESS.2023.3270417>
- Applegate, D. L., Bixby, R. E., Chvátal, V., & Cook, W. J. (2006). *The traveling salesman problem: A computational study*. Princeton University Press.
- de Oliveira, F. B., Silva, D. R., & de Lima, B. S. (2021). A study on the performance of ant colony optimization for the traveling salesman problem. *Applied Soft Computing*, 103, 107159. <https://doi.org/10.1016/j.asoc.2021.107159>
- Dorigo, M., & Blum, C. (2005). Ant colony optimization theory: A survey. *Theoretical Computer Science*, 344(2–3), 243–278. <https://doi.org/10.1016/j.tcs.2005.05.020>
- Dorigo, M., & Gambardella, L. M. (1997). Ant colonies for the traveling salesman problem. *BioSystems*, 43(2), 73–81. [https://doi.org/10.1016/S0303-2647\(97\)01708-5](https://doi.org/10.1016/S0303-2647(97)01708-5)
- Dorigo, M., & Gambardella, L. M. (1997). Ant colony system: A cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*, 1(1), 53–66. <https://doi.org/10.1109/4235.585892>
- Dorigo, M., Maniezzo, V., & Colomi, A. (1996). Ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 26(1), 29–41. <https://doi.org/10.1109/3477.484436>
- Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. W. H. Freeman.
- Hamza, A., Benkhelifa, E., & Alharbi, Y. (2024). Performance analysis of ant colony optimization parameters in large-scale optimization problems. *Expert Systems with Applications*, 238, 121971. <https://doi.org/10.1016/j.eswa.2023.121971>
- Hao, T., Zhang, Y., Liu, X., & Chen, Z. (2023). Study on a hybrid algorithm combining enhanced ant colony optimization and double improved simulated annealing via clustering in the traveling salesman problem. *PeerJ Computer Science*, 9, e1609. <https://doi.org/10.7717/peerj-cs.1609>
- Hussein, M. (2020). Meta-heuristics explained: Ant colony optimization. Medium. <https://readmedium.com/meta-heuristics-explained-ant-colony-optimization-d016fe925108>
- Lawler, E. L., Lenstra, J. K., Rinnooy Kan, A. H. G., & Shmoys, D. B. (1985). *The traveling salesman problem: A guided tour of combinatorial optimization*. Wiley.
- Marín, Y. (2020). aco-tsp (Version commit 5ea3600) [Computer software]. GitHub. <https://github.com/yammadev/aco-tsp>
- Miranda, L. J. (2017, January 18). Solving the travelling salesman problem using ant colony optimization. LJ Miranda. <https://lvmiranda921.github.io/notebook/2017/01/18/ant-colony-optimization-tsp/>
- Nallaperuma, S., Neumann, F., & Pappa, G. L. (2015). An experimental analysis of ant colony optimization algorithms for the traveling salesman problem. *Evolutionary Computation*, 23(4), 621–655. https://doi.org/10.1162/EVCO_a_00144
- Purkayastha, A. A., Tharwani, J., & Aggarwal, S. (2025). Ant colony optimization for CMOS physical design: Reducing layout area and improving aspect ratio in VLSI circuits. *Electronics*, 14(24), 4825. <https://www.mdpi.com/2079-9292/14/24/4825>
- Shen, S.-M., Liu, H.-M., Zhan, H.-X., Liu, Z.-F., Guo, G., Yu, C., & Hu, P.-C. (2025). Efficient maximum iterations for swarm intelligence algorithms: A comparative study. *Artificial Intelligence Review*. <https://doi.org/10.1007/s10462-024-10833-1>

Shi, Y., Wang, L., & Zhang, J. (2022). Parameter tuning of ant colony optimization using uniform experimental design. *IEEE Access*, 10, 87432–87444. <https://doi.org/10.1109/ACCESS.2022.3198915>

Skinderowicz, R. (2022). Improving ant colony optimization efficiency for solving large TSP instances. *Applied Soft Computing*, 120, 108653. <https://doi.org/10.1016/j.asoc.2022.108653>

Stützle, T., & Dorigo, M. (1999). ACO algorithms for the traveling salesman problem. In *Evolutionary algorithms in engineering and computer science* (pp. 163–183). Wiley.

Stützle, T., & Hoos, H. H. (2000). MAX–MIN ant system. *Future Generation Computer Systems*, 16(8), 889–914. [https://doi.org/10.1016/S0167-739X\(00\)00043-1](https://doi.org/10.1016/S0167-739X(00)00043-1)

Ullrich, C., Neumann, F., & Witt, C. (2024). Experimental analysis of pheromone evaporation in ant colony optimization. *Scientific Reports*, 14, 4121. <https://doi.org/10.1038/s41598-024-64121-5>

Wang, H., Zariphopoulou, T., & Zhou, X. (2018). Exploration versus exploitation in reinforcement learning: A stochastic control approach. *arXiv*. <https://arxiv.org/abs/1812.01552>

Wang, Y., & Han, Z. (2021). Ant colony optimization for traveling salesman problem based on parameters optimization. *Applied Soft Computing*, 112, 107738. <https://doi.org/10.1016/j.asoc.2021.107439>

Zarrinpanjeh, N., Samadzadegan, F., & Reinartz, P. (2020). A study of ant colony optimization parameters on shortest path problems. *ISPRS International Journal of Geo-Information*, 9(5), 300. <https://doi.org/10.3390/ijgi9050300>